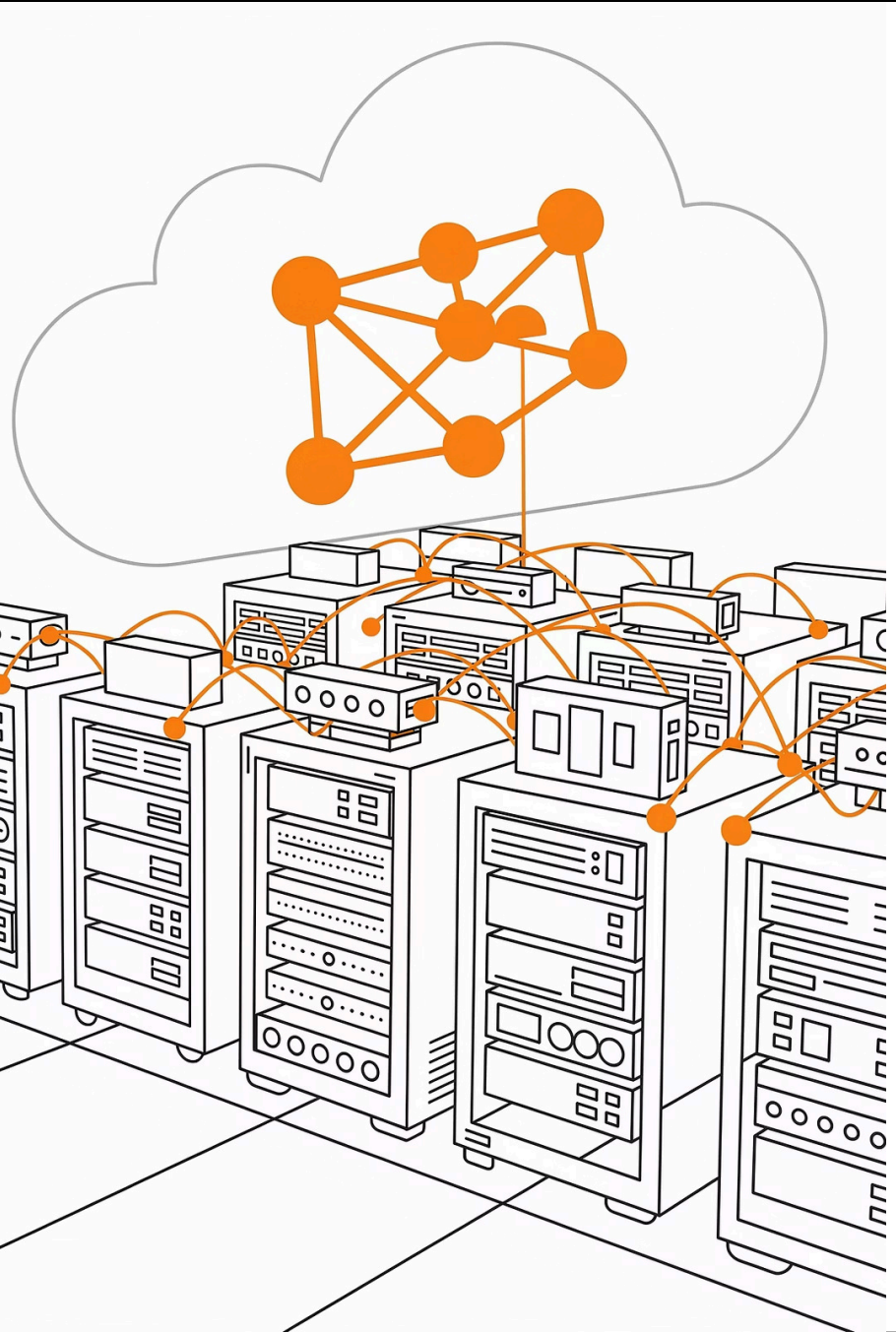


Module 3

RAG on Bedrock

Current State — February 2026 | Amazon Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases Overview

Amazon Bedrock Knowledge Bases is a fully managed RAG (Retrieval Augmented Generation) capability that orchestrates data ingestion, retrieval, and generation across supported data sources and vector stores — all without requiring teams to build and maintain a custom pipeline.

Managed Connectors

Native integrations for listed data sources, plus programmatic ingestion APIs for custom sources.

Auto Embeddings

Automatically converts ingested documents into vector embeddings using your selected embedding model.

Multi-Source Support

S3, Confluence, SharePoint, Salesforce, Web Crawler (preview), and more via configuration.

Vector DB Flexibility

Choose from AWS-native or third-party managed vector stores supported by Knowledge Bases.

Built-in Security

IAM integration, encryption at rest and in transit, VPC support, and AWS-native security controls.

RAG Architecture: Data Sources (2026)

Unstructured — Native Connectors

→ Amazon S3

PDFs, HTML, Office docs. Incremental sync, metadata fields, prefix/suffix filters.

→ Confluence

Enterprise wiki content. Scheduled or on-demand sync via native connector.

→ SharePoint

Document libraries and sites with permission-aware access.

→ Salesforce

CRM records and knowledge articles, respecting object and field-level permissions.

→ Web Crawler (Preview)

Crawl public web content with configurable depth, domain restrictions, and URL filtering.

Structured / Graph / Enterprise Search

Amazon Kendra / GenAI Index

Leverage existing enterprise search indexes as a Knowledge Bases retrieval backend. Enables hybrid keyword + semantic search over enterprise content.

Amazon Neptune Analytics

Build **GraphRAG** solutions on graph-structured data — multi-hop traversal, entity linking, and relationship-aware retrieval.

Natural Language → SQL

Supported as an **agent pattern** via Agents + Lambda + SQL endpoints (RDS/Aurora). Not a direct KB data source type.

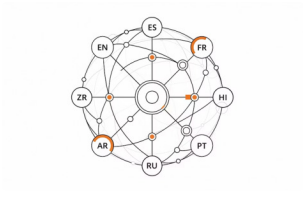
For sources not natively supported, use programmatic ingestion APIs to push pre-processed documents and embeddings directly into a Knowledge Base.

Embedding Models

The choice of embedding model directly impacts retrieval quality, latency, and cost. Bedrock Knowledge Bases supports Amazon-native, Cohere, and custom/third-party models to address diverse use cases.

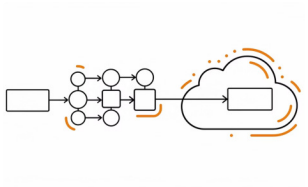
Amazon Titan Embeddings

- Titan Text Embeddings for text-only content
- Titan Multimodal Embeddings for text + image in multimodal Knowledge Bases
- ~$0.0001 per 1K tokens (region-dependent; see pricing page)
- Best for general-purpose, cost-effective semantic search



Cohere Embed Models

- English-focused and multilingual models via Bedrock
- 100+ language support; dimensions range 768–4096
- Well-suited for cross-lingual and international applications



Custom & Third-Party Embeddings

- Run custom models on SageMaker endpoints
- Generate embeddings externally and ingest via programmatic APIs
- Hugging Face and other sources integrated via SageMaker — not available as direct drop-down options in the KB UI

Vector Databases: Supported Options

Knowledge Bases supports multiple AWS-native and third-party vector stores as retrieval backends. The right choice depends on your existing infrastructure, workload profile, and operational preferences.

Amazon OpenSearch Serverless

Fully managed, no cluster sizing. Auto-scaling OCUs for variable workloads. k-NN ANN retrieval + hybrid keyword/semantic search. **Best for:** AWS-native deployments with fluctuating query volumes.

Amazon OpenSearch Service (Managed Clusters)

Fine-grained control over cluster config and performance tuning. Access to advanced ML features. **Best for:** Enterprises already running OpenSearch who need tuning control.

Amazon Aurora PostgreSQL (pgvector)

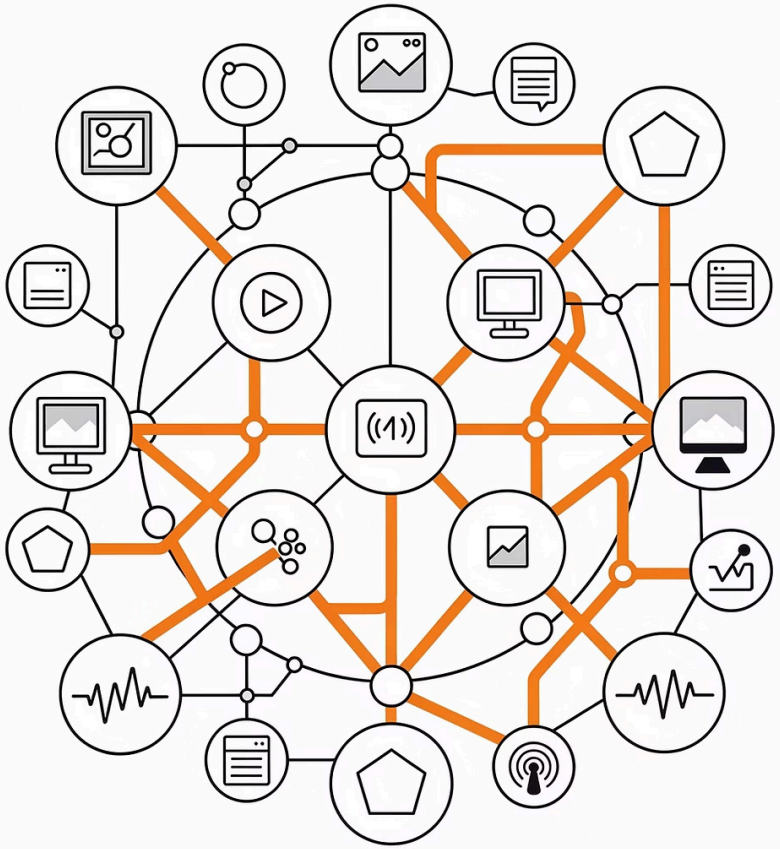
Relational + vector via pgvector extension. ACID transactions and SQL queries alongside embeddings. **Best for:** Teams already on PostgreSQL who want to add vector search.

Amazon Neptune Analytics

Graph-native store for GraphRAG. Multi-hop graph traversal and relationship-aware retrieval. **Best for:** Knowledge graphs and complex entity relationships.

Third-Party Stores

Pinecone — purpose-built, high-performance vector search at scale. **Redis Enterprise Cloud** — low-latency in-memory for real-time use cases. **MongoDB Atlas** — document + vector with flexible schema and global distribution.



Advanced RAG Features (2026)

GraphRAG (Neptune)

Multi-hop reasoning across entities and relationships. Better entity linking, disambiguation, and explainable retrieval paths. Use cases: fraud networks, biomedical knowledge, complex document interpretation.

Multimodal RAG

Retrieve and ground answers on text, images, audio, video, and document layouts using Titan Multimodal Embeddings + Amazon Nova. Handles annotated docs, diagrams, charts, and slide decks.

Reranking Models

A reranker model scores candidate chunks against the query, reordering the top-k results for higher precision. Use Cohere rerankers or LLM-based rerank patterns. Particularly valuable for ambiguous queries.

Hybrid Search

Combines keyword/lexical and semantic vector search. Better recall on specific terms, IDs, and structured text. Natively supported in OpenSearch (serverless and managed) and via Kendra integration.

Chunking Strategies

How you split documents into chunks is one of the most impactful decisions in a RAG pipeline — it determines context coherence, retrieval precision, and ultimately generation quality.

Default / Automatic Chunking

Knowledge Bases automatically parses and chunks documents using a fixed-size with overlap strategy — roughly in the **few-hundred token range** with configurable overlap.

Effective for most generic use cases out of the box with minimal configuration required.

Advanced Chunking Options

1 Semantic Chunking

Split along semantic units — paragraphs, sections, headings — to preserve coherent context within each chunk.

2 Hierarchical Chunking

Represent parent-child document structure (document → section → paragraph) via metadata or graph structures for context-aware retrieval.

3 Custom Chunking

Pre-process documents via Lambda or offline scripts, then push via programmatic ingestion APIs. Full control over split logic.

4 Metadata Preservation

Maintain document title, section, page number, language, and version with each chunk for richer filtering and interpretability.

RAG APIs

Knowledge Bases exposes two core APIs that support different levels of control — from pure retrieval to fully managed end-to-end RAG in a single call.

Retrieve API

Purpose: Retrieve relevant chunks or documents *without* running generation — giving you full control over the generation step.

When to use:

- Custom or model-specific generation logic outside Knowledge Bases
- Hybrid systems where retrieval and generation are decoupled
- Debugging retrieval quality and coverage independently

Returns: Ranked list of retrieved contexts with relevance scores, source metadata, and multimodal references (when enabled).

RetrieveAndGenerate API

Purpose: End-to-end RAG in a single API call — retrieval plus generation in one managed workflow.

When to use:

- Rapid implementation of Q&A bots, assistants, and knowledge agents
- Standardized RAG flows requiring minimal orchestration code

Features: Custom prompt templates, selectable foundation model, configurable RAG parameters, and grounded responses with citations (text and multimodal).

Use the **Retrieve API** when you need flexibility and debugging control. Use **RetrieveAndGenerate** when you want a production-ready, low-code RAG endpoint.

RAG Evaluation (2026)

Knowledge Bases includes built-in **RAG evaluation** features covering both retrieval-only and end-to-end retrieval + generation scenarios, powered by LLM-as-a-Judge scoring.

Retrieval Evaluation Metrics

Context Relevance

Are retrieved chunks relevant to the user's query?

Coverage

Do retrieved chunks collectively cover the important aspects of the query?

Used to tune chunking strategy, embedding model selection, top-k, filters, and hybrid search parameters.

Generation Evaluation Metrics

Quality

Correctness vs. ground truth, **Completeness** of the answer, and **Faithfulness/Groundedness** — is the answer supported by retrieved context?

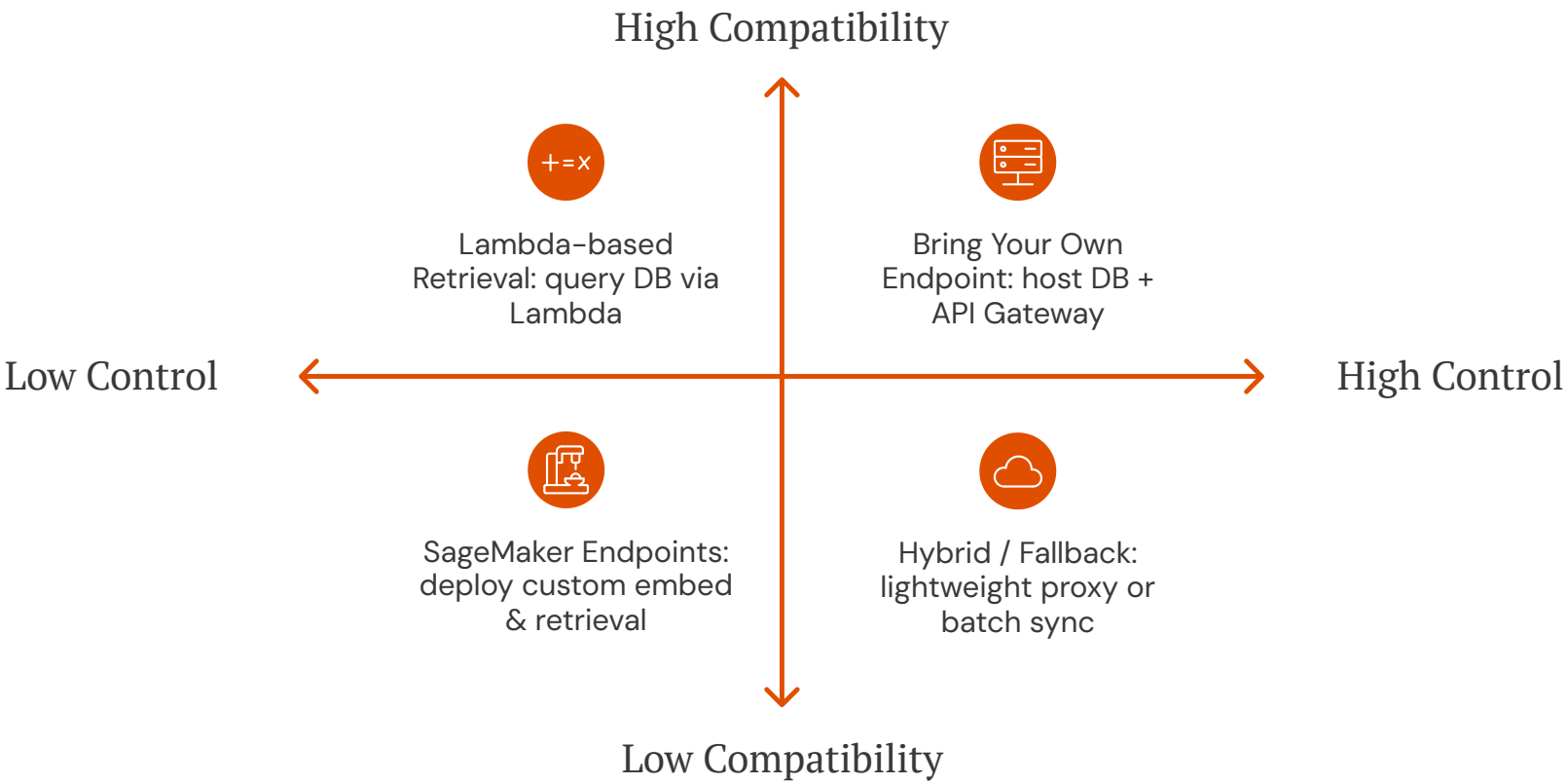
Responsible AI

Harmfulness detection, **Appropriate refusal** behavior, and **Stereotyping/bias** checks.

Evaluator models include Anthropic Claude, Amazon, and Meta models. Metrics and prompts are configurable for domain and risk tolerance.

Integrating Vector DBs Not Natively Supported

Knowledge Bases maintains a **fixed list of supported vector stores**. For external databases not on that list, teams use custom architectures that pair Bedrock models with any vector DB of their choice.



These patterns do not register the external store inside Knowledge Bases directly, but they fully enable RAG architectures using Bedrock inference APIs (Converse, InvokeModel, Responses) with any vector database — at the cost of additional custom code and operational overhead compared to the managed KB store list.

Improving RAG Quality

Embedding Model Selection

Domain-tuned models can improve recall/precision by **20–40%** per community benchmarks. Consider domain specificity, multilingual needs, and dimension size vs. cost trade-offs. Best practice: run small evaluations on your own corpus using built-in RAG evaluation tools.

Retrieval Optimization

Tune top-k, similarity thresholds, and hybrid weights. Apply **metadata filters** (doc type, product, region) to narrow search space. Introduce reranking on the candidate set to improve final ordering and precision.

Prompt Engineering

Use system prompts to constrain tone and grounding ("use only provided context"). Add few-shot examples demonstrating ideal citation behavior. Keep prompts concise but include explicit citation requirements.

Chunking Strategy

300–500 token chunks with 10–20% overlap is a strong starting heuristic. Split along semantic boundaries. Propagate key metadata (title, section, page, language, version) with every chunk.

Data Quality

Remove boilerplate, duplicates, and stale content. Maintain structured metadata and consistent tagging. Schedule regular syncs and use versioning for corpus A/B testing and rollback capability.

Cost Optimization for RAG

RAG workloads on Bedrock incur costs across three primary dimensions: **storage**, **embedding/ingestion**, and **per-query retrieval + generation**. Understanding each layer is essential to managing total cost of ownership.

Cost Dimensions

Vector Storage

OpenSearch Serverless: OCUs + storage. Aurora: instances + storage + I/O. Third-party (Pinecone, Redis, MongoDB): vendor pricing per index, vector count, and capacity.

Embedding / Ingestion

One-time cost per document/chunk at ingestion time. Use incremental sync to pay only for new/updated docs. For large backfills, use batch inference pricing.

Per-Query Costs

Vector-store query + optional reranking model inference + generation FM cost. Tune top-k, use filters, and restrict multimodal retrieval where not needed.

Optimization Strategies

→ Cache Frequent Queries

Implement response caching at the app or API layer. Use Bedrock prompt caching for repeated prompt prefixes on supported models.

→ Right-Size Chunks & Top-k

Optimize chunk size and retrieved chunk count to hit acceptable quality at minimum token and compute cost.

→ Choose Cost-Efficient Embeddings

Use smaller or cheaper embedding models where quality remains sufficient for your use case.

→ Batch Document Updates

Group re-embedding jobs to leverage batch inference pricing and reduce per-call overhead.

→ Archive Cold Data

Maintain only necessary embeddings in the active vector store — archive or remove low-value or stale content.

Key Takeaways for Workshop Content

01

Managed RAG Benefits

Show how Knowledge Bases handles connectors, ingestion, embeddings, vector storage, retrieval, and RetrieveAndGenerate — so teams can focus entirely on business logic rather than pipeline plumbing.

02

Vector Store Selection

Build a decision matrix comparing OpenSearch (Serverless vs. Managed), Aurora pgvector, Neptune Analytics, and third-party stores across latency, scale, cost, and existing infrastructure investments.

03

Embedding Model Impact

Demonstrate retrieval quality differences across embedding models on the same corpus using built-in RAG evaluation tools. Make the performance vs. cost trade-off tangible with real numbers.

04

Evaluation Framework

Use Knowledge Bases evaluation (LLM-as-a-Judge) to teach participants how to measure and iteratively improve both retrieval precision and generation quality with configurable metrics.

05

Cost Awareness

Break down storage, embedding, retrieval, and generation costs line by line. Show how configuration choices — chunk size, top-k, model selection — directly impact the monthly bill.